

Design and Deployment of an Event-Driven Microservices Architecture Using the Strangler Pattern and Cloud-Native Technologies

Suyash Sharma (40317581)
Department of Electrical and Computer Engineering
Concordia University, Montreal, Canada
Email: s_suya@live.concordia.ca

Abstract—This project presents the design and deployment of a cloud-native microservices architecture built using FastAPI, Docker, MongoDB, and RabbitMQ. The system contains two versions of a User microservice (v1 and v2) and an Order microservice, all orchestrated through an API Gateway implementing the Strangler Pattern. A key requirement is the automatic synchronization of user data—specifically email and delivery address—across both the User and Order databases. The project further includes an event-driven mechanism using RabbitMQ to propagate updates, version-based routing controlled by a configuration file, and deployment on a cloud virtual machine using Docker. Screenshots, database samples, and workflow demonstrations are included throughout the report.

Index Terms—Microservices, Strangler Pattern, Event-Driven Architecture, FastAPI, Docker, MongoDB, RabbitMQ, JWT Authentication, Cloud Deployment

I. INTRODUCTION

Microservices have become a standard approach for designing scalable and modular cloud-native applications. They allow independent deployment, versioning, and scaling of system components. However, migrating from an older service to an improved version introduces challenges, including state consistency, routing, and minimizing downtime.

This project develops a fully functional event-driven system consisting of:

- A **User Microservice v1** (basic CRUD operations).
- A **User Microservice v2** (refactored version with JWT authentication and improved structure).
- An **Order Microservice** storing orders linked to the user's email and delivery address.
- An **API Gateway** that routes traffic using the **Strangler Pattern**.
- A **RabbitMQ event-driven system** used to synchronize email and delivery address updates across services.

MongoDB serves as the persistent storage for both User and Order databases as well as the event log. Docker is used to containerize all microservices and RabbitMQ, and the full system is deployed on AWS to expose public API endpoint.

This architecture demonstrates modular service design, database synchronization through events, version migration, and cloud deployment in a production-like environment.

II. MICROSERVICE DESIGN

A. User Service (v1 and v2)

The User Service maintains:

- User ID
- Name
- Email
- Delivery Address
- Password (v2 only, hashed)

Version v2 adds JWT authentication and refactored code while maintaining backward compatibility.

Requirement (1) states that the User DB must contain at least user ID, email, and delivery address, which our MongoDB implementation satisfies.

Screenshot 1: POST /users (User Creation)

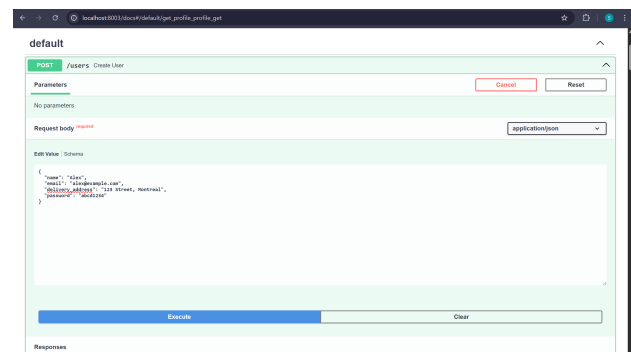


Fig. 1: Creating a new user using Swagger UI.

Screenshot 2: User Document in MongoDB

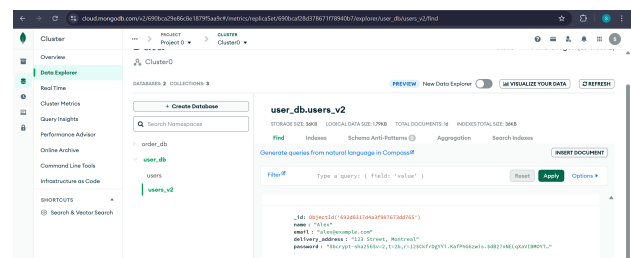


Fig. 2: User document stored in MongoDB.

B. Order Service

The Order Service stores:

- Order ID
- Items Ordered
- User Email (copied at order creation time)
- Delivery Address (copied from user profile)
- Status: under process, shipping, delivered

This satisfies Requirement (1): order DB mirrors email + delivery address from the User DB.

Screenshot 3: POST /orders (Order Creation)

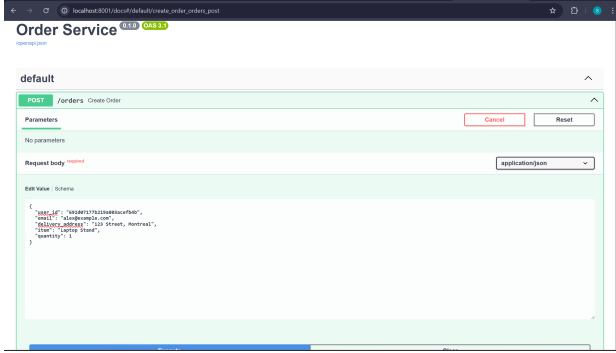


Fig. 3: Creating an order with user email + address.

Screenshot 4: Order Document in MongoDB

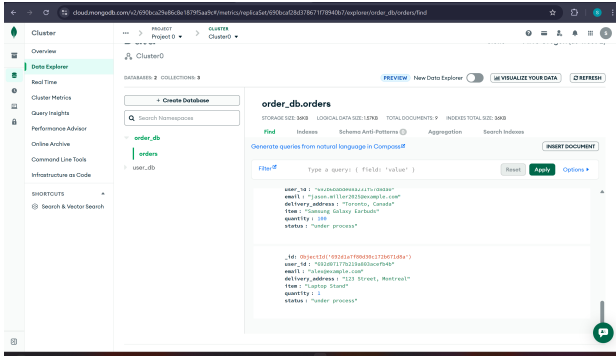


Fig. 4: Stored order in MongoDB.

C. Data Model and Database Schema

The system uses MongoDB as the persistent storage backend for all microservices. Each service maintains its own collection, ensuring loose coupling and independent scalability. The following schemas define the core data models.

1) *User Collection (User Service)*: The User database stores account information and fields required for synchronization with the Order Service. The schema is:

```
User {
  _id: ObjectId
  name: string
  email: string
  delivery_address: string
  password: string (hashed, User v2 only)
```

```
}
```

This collection is the authoritative source for email and delivery address. Any update triggers an event to RabbitMQ for synchronization.

2) *Order Collection (Order Service)*: Every order stores a snapshot of the user's email and delivery address at the time of ordering. These fields remain synchronized when the user updates their profile.

```
Order {
  _id: ObjectId
  user_id: string
  item: string
  quantity: integer
  email: string
  delivery_address: string
  status: string
    ("under process", "shipping", "delivered")
}
```

3) *Event Collection (Event Log)*: Each address or email update results in an event consumed by the Order Service and stored for debugging and traceability.

```
Event {
  event_id: string
  user_id: string
  updated_fields: {
    email: string (optional)
    delivery_address: string (optional)
  }
  timestamp: datetime
}
```

This event-driven model ensures eventual consistency across microservices without tight coupling.

D. API Gateway and Routing (Requirement 2 & 5)

The API Gateway:

- Routes requests to microservices based on endpoint.
- Supports user creation, updates, and order operations through HTTP/REST.
- Implements Strangler Pattern by routing:

$$P\% \rightarrow \text{User v1} \quad \text{and} \quad (1 - P)\% \rightarrow \text{User v2}$$

- Reads P dynamically from a configuration file (gateway_config.json), not hardcoded.

API Design Overview

The API Gateway provides a unified REST interface for all microservices. It abstracts internal service endpoints and exposes a clean, consistent API surface to the client. The design follows RESTful principles using JSON request/response formats.

1. *User Service API Endpoints (v1 and v2)*: API operations for user creation, profile updates, and authentication.

TABLE I: User Service API Endpoints

Method	Endpoint	Description
POST	/users	Create a new user (name, email, delivery address, password).
PUT	/users/{id}	Update email or delivery address.
POST	/login	User authentication (v2 only) — returns JWT token.
GET	/profile	Protected endpoint returning user info using JWT token.

TABLE II: Order Service API Endpoints

Method	Endpoint	Description
POST	/orders	Create a new order with item, quantity, email, and address.
GET	/orders	Retrieve all orders or filter by status.
PUT	/orders/{id}	Update order status.
PUT	/orders/update-user-info	Sync email or address from event update.

2. *Order Service API Endpoints*:

3. *API Gateway Routing Rules*: The API Gateway forwards incoming HTTP requests to the appropriate microservice based on:

- Request path - HTTP method - Strangler Pattern percent-age split (v1 vs v2)

TABLE III: Gateway Routing Rules

Incoming Request	Forwarded To	Notes
POST /users	User v1 or v2	Based on P split ratio
PUT /users/{id}	User v1 or v2	Based on P split ratio
POST /login	User v2 only	JWT authentication implemented only in v2
/order APIs	Order Service	Directly forwarded

This API design ensures a clean separation of concerns, simplifies client interaction, and supports progressive migration under the Strangler Pattern.

Screenshot : gateway_config.json (Traffic Split Configuration)

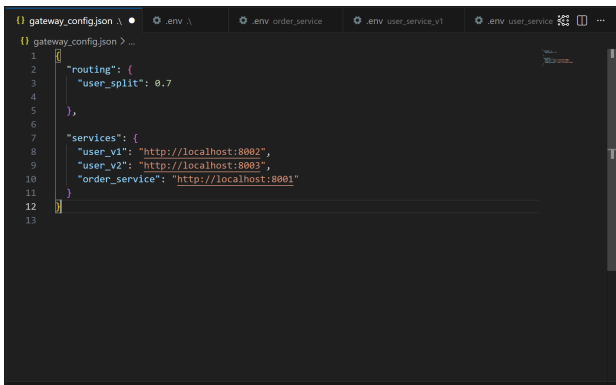


Fig. 5: Configuration file controlling percentage routing for v1 and v2.

This satisfies Requirement (5).

E. *Event-Driven Synchronization (Requirement 3)*

When the user updates their email or delivery address via a PUT request:

1. User Service publishes an event to RabbitMQ. 2. The Order Service consumes the event. 3. MongoDB Order documents are automatically updated.

Screenshot 5: Updating User Address (PUT)

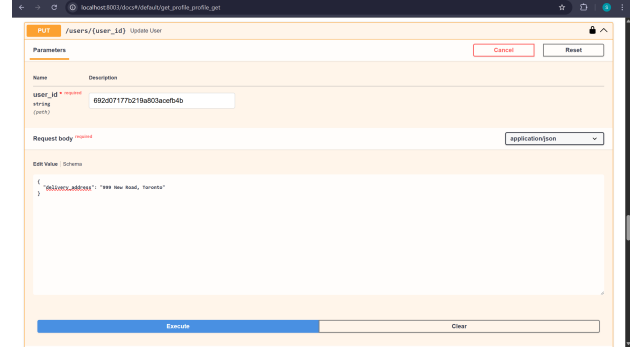


Fig. 6: PUT request updating email/address.

Screenshot 6: New Order Showing Updated Address

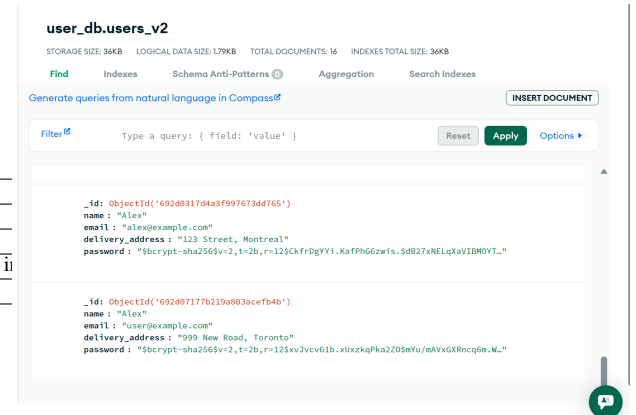


Fig. 7: Order reflects synchronized address after event.

Screenshot 7: RabbitMQ Event Queue Showing Address Update Message

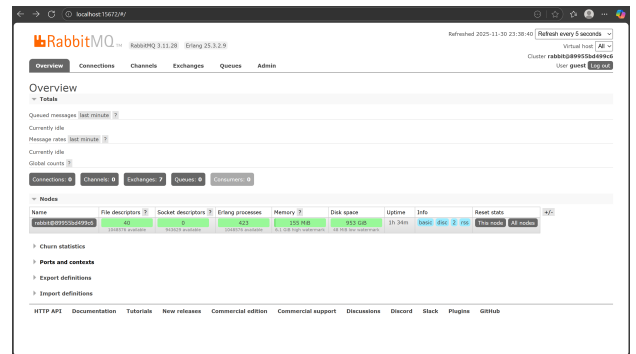


Fig. 8: RabbitMQ queue displaying published user-update event.

Screenshot 7: RabbitMQ Event Queue Showing Address Update Message

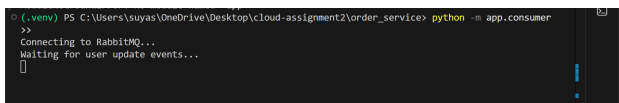


Fig. 9: RabbitMQ queue displaying published user-update event.

F. Version Introduction (Requirement 4)

User v2 introduces:

- JWT login
- Cleaner models
- Improved hashing
- Authentication middleware

The gateway continues sending a configurable percentage of traffic to v1.

Screenshot : Refactored User Service v2

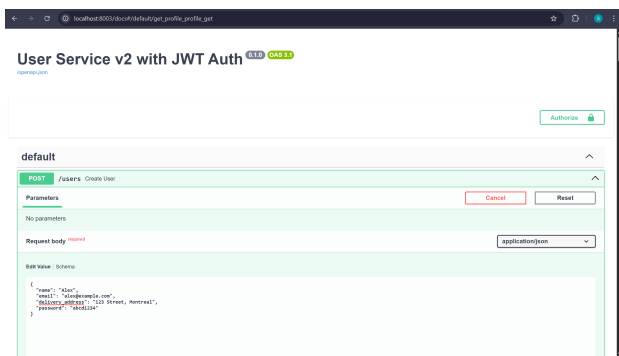


Fig. 10: Refactored User Microservice v2 showing updated implementation.

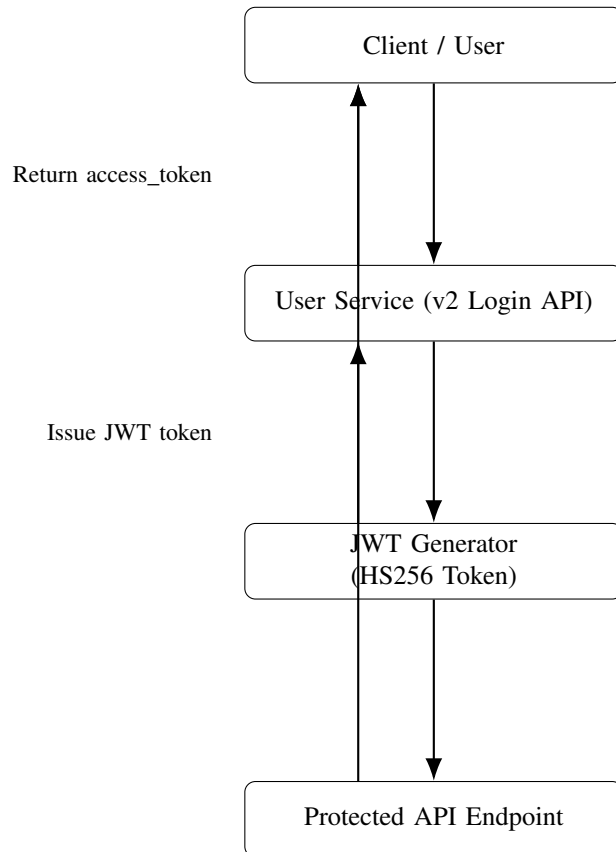


Fig. 11: JWT Authentication Flow: Token generation and validation across login and protected endpoints.

Screenshot : JWT_Token Response

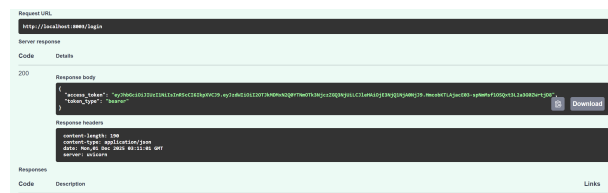


Fig. 12: JWT access_token generated after successful user login.

Screenshot : Authenticated JWT Request

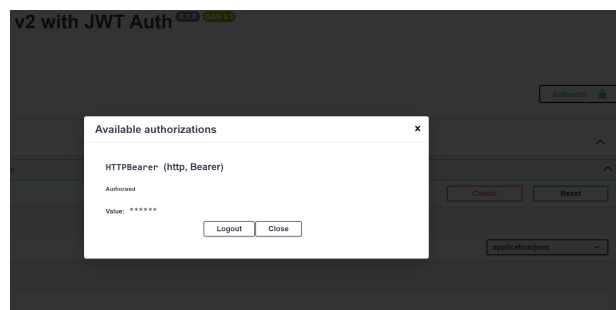


Fig. 13: Using Bearer JWT token to access the protected /profile endpoint.

III. IMPLEMENTATION

A. Database Layer (Requirement 6)

Both User and Order databases use **MongoDB**. The event-driven data (email/address change events) is also stored in MongoDB for debugging and traceability.

B. Event-Driven System (RabbitMQ)

RabbitMQ handles:

- User update events
- Order service synchronization
- Asynchronous communication

C. Deployment Using Docker (Requirement 7)

I used Monolithic deployment option

- **Monolith deployment:** all services + gateway + RabbitMQ in one docker-compose.
- **Microservice deployment:** each service has its own Docker container.

Screenshot : Docker_compose

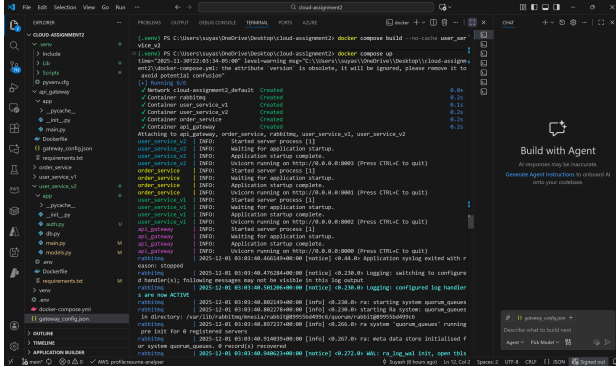


Fig. 14: Docker Compose file showing all services packaged together in the monolithic deployment.

Screenshot : Docker_engine

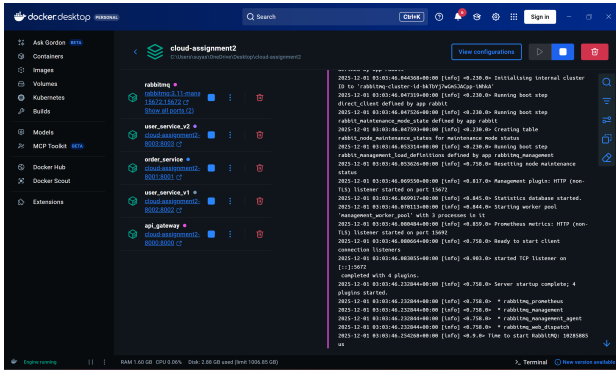


Fig. 15: Docker Engine showing all running containers for the monolithic deployment.

D. Cloud VM Deployment (Requirement 8)

The entire system was launched on an online VM. All services were exposed publicly through VM endpoints:

- API Gateway
- User v1 service
- User v2 service
- Order service
- RabbitMQ management console

Screenshot : AWS_ECS Deployment

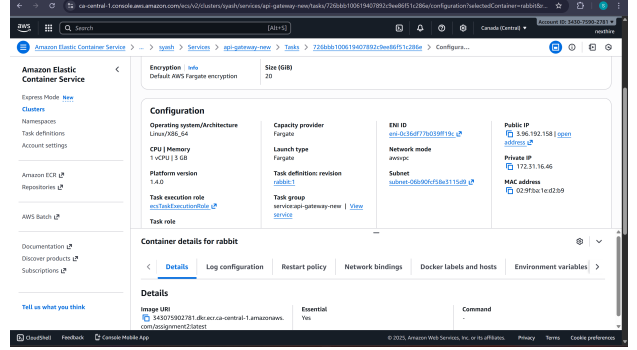


Fig. 16: AWS ECS dashboard showing running services and deployed containers.

Public Endpoint (VM IP): 3.96.192.158 Screenshot : Gateway Running on Public IP

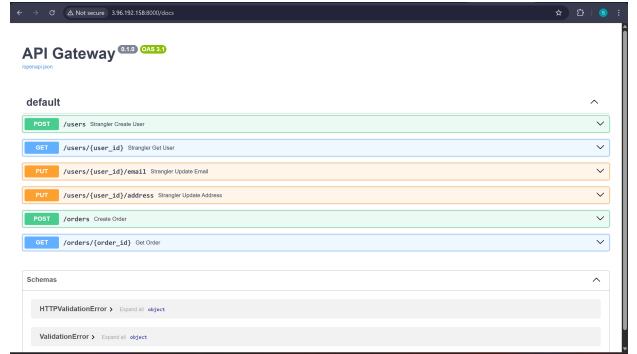


Fig. 17: API Gateway successfully running and accessible via the public VM endpoint.

ACKNOWLEDGMENT

This project was fully implemented and documented by **Suyash Sharma**. I would like to thank Professor Yan Liu Yan.liu@concordia.ca Professor, PhD. PEng, Gina Cody Research and Innovation Fellow for guidance and Concordia University cloud infrastructure support.

MEMBER CONTRIBUTION

TABLE IV: Member Contribution Summary

Name (SID)	Task List	Contribution (%)
Suyash Sharma (40317581)	All Tasks (1–8)	100%

REFERENCES

- [1] FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python 3.7+,” Available: <https://fastapi.tiangolo.com/>
- [2] RabbitMQ Documentation Team, “RabbitMQ: Messaging that Just Works – Official Guide and Tutorials,” Available: <https://www.rabbitmq.com/documentation.html>
- [3] Docker Inc., “Docker Documentation: Build, Ship, and Run Distributed Applications,” Available: <https://docs.docker.com/>
- [4] Amazon Web Services, “Amazon ECS, Fargate, and ECR Developer Guide – Deploying Containerized Microservices on AWS,” Available: <https://docs.aws.amazon.com/>
- [5] MongoDB Inc., “MongoDB Documentation: NoSQL Database for Modern Applications,” Available: <https://www.mongodb.com/docs/>
- [6] Martin Fowler, “Strangler Fig Application Pattern,” Available: <https://martinfowler.com/bliki/StranglerFigApplication.html>